

Controllers, Services, Repositories, Middlewares & Request Context — Lecture Notes

1. The Request Lifecycle Inside the Server

When a client sends an HTTP request, the journey *inside* the server follows a well-defined path:



This three-layer separation (Controller → Service → Repository) is a **design pattern**, not a hard requirement. A single function could do all of it — but separation makes code scalable, maintainable, and easier to debug.

2. The Three Layers

2.1 Controller / Handler

Receives: `request` object + `response` object (provided by the framework/runtime).

Responsibilities — in order:

1. **Binding / Deserialization** — Extract and deserialize data from the request into the language's native format.
 - GET → extract query params and path params
 - POST/PATCH/PUT → extract and deserialize request body (JSON → struct/dict/class)
 - In Node.js, a body-parser middleware often handles this upstream; in Go/Python/Rust it must be done explicitly.
 - If deserialization fails → return `400 Bad Request`, terminate immediately.
2. **Validation & Transformation** — Validate the deserialized data; transform it for downstream convenience (e.g., set defaults for missing optional query params, cast types, normalize values).
 - If validation fails → return `400 Bad Request`, terminate immediately.
3. **Call the Service Layer** — Pass validated + transformed data (plus auth context like `userId`, `role`) to the service method.
4. **Send Response** — Based on the service result, decide the appropriate status code and shape the response body.
 - Success: `200`, `201`, `204`
 - Client error: `4xx`
 - Server error: `500`

Key rule: Controllers deal with HTTP concerns only — status codes, request/response format, validation. No business logic lives here.

2.2 Service Layer

Receives: Native data from the controller (no HTTP objects — no request/response).

Responsibilities:

- Contains all **business logic** for the API — the actual processing.
- Orchestrates multiple repository calls, merges data, applies rules.
- Makes external calls: emails, notifications, webhooks, third-party APIs.
- Returns a result (data or error) to the controller — it never decides HTTP status codes.

Key rule: A service method should look like a plain function — you should not be able to tell from reading it that it's part of an HTTP API. No HTTP concepts leak into this layer.

Service can:

- Call `repo.getAllBooks(sort)`
- Call `repo.getUserById(userId)`
- Merge results
- Send an email

```
└─ Return { books: [...] }
```

Service cannot:

```
└─ Access req / res objects
└─ Set HTTP status codes
```

2.3 Repository Layer

Receives: Filtered, sorted, ready-to-use data from the service layer.

Responsibilities:

- Sole responsibility: **construct and execute database queries**, return raw results.
- Each method does exactly **one thing** — fetch books, insert a book, delete by ID.

Key rule: One repository method = one data operation. Never use optional parameters to make one function do two different things.

```
✓ getBooks(sort, limit, offset) → returns all books
✓ getBookById(id)                → returns one book
✗ getBooks(id?)                  → returns all OR one based on id
```

2.4 Layer Responsibilities at a Glance

Layer	Deals With	Does NOT deal with
Controller	HTTP: request parsing, validation, response codes	Business logic, DB queries
Service	Business logic, orchestration, external calls	HTTP concepts, raw DB queries
Repository	DB queries: insert, fetch, update, delete	Business logic, HTTP concepts

3. Middlewares

What is a Middleware?

A function that executes **at a boundary** in the request lifecycle — before routing, between routing and handler, or after handler — that can:

- Read and modify the **request** object
- Read and modify the **response** object

- Pass execution to the next step via `next()`
- Or **short-circuit** by sending a response directly (terminating the request early)

Signature: every middleware receives three things:

```
(request, response, next)
```

- `next()` — passes execution to the next middleware or the next execution context (routing, handler, etc.)
- If `next()` is not called and no response is sent → request hangs.

Middlewares are **optional** — they may or may not be present depending on requirements.

Why Middlewares?

Same reason we use functions — **avoid code duplication**. Common operations that need to run for every (or many) API requests shouldn't be copy-pasted into every handler. Delegate them to middleware.

Two requirements for something to be a middleware:

1. The logic needs to run for **multiple** (or all) requests.
 2. It needs access to the `request` and/or `response` object.
-

Common Middleware Examples

CORS Middleware

- Reads `Origin` header from request.
- If origin is in the allowed list → adds `Access-Control-Allow-Origin` and related headers to response.
- Calls `next()` to continue.
- If origin not allowed → doesn't add headers; browser will block the response.
- **Why middleware:** Needs to run for every request and modify response headers.

Security Headers Middleware

- Adds headers like `Content-Security-Policy`, `X-Frame-Options`, `Strict-Transport-Security` to every response.
- Calls `next()` to continue.

Authentication Middleware

- Extracts token (JWT or session ID) from request headers or cookies.

- Verifies the token.
 - **Failure** → sends `401 Unauthorized`, terminates request immediately (no `next()` call).
 - **Success** → extracts `userId`, `role`, permissions; stores them in **request context**; calls `next()`.
- **Why middleware:** Authentication must run for most protected routes without duplicating it in every handler.

Rate Limiting Middleware

- Checks how many requests this IP has made in a recent time window.
- Exceeds threshold → `429 Too Many Requests`, terminate.
- Under threshold → `next()`.

Logging / Monitoring Middleware

- Logs request method, path, query params, body, timestamp for every request.
- Always calls `next()`.
- Helps with debugging, auditing, and performance monitoring.

Global Error Handling Middleware

- Placed **last** in the middleware chain.
- Catches any unhandled error from any point in the lifecycle (handler, service, other middlewares).
- Determines whether it's a client error (4xx) or server error (5xx).
- Sends a properly structured error response: `{ message, code, status }`.

Ordering matters: Errors flow forward, not backward. If the error middleware is placed in the middle, errors from handlers further down won't reach it.

Typical Middleware Order

- | | |
|---|-----------------------------------|
| 1. CORS | ← terminate unknown origins early |
| 2. Security Headers | ← set on every response |
| 3. Logging | ← record every request |
| 4. Authentication | ← verify identity; set context |
| 5. Rate Limiting | ← protect server resources |
| 6. [Routing → Handler → Service → Repository] | |
| 7. Global Error Handler | ← always last |

4. Request Context

What is it?

A **per-request shared state** — a key-value store that is:

- Scoped to a single HTTP request (not shared across requests)
- Accessible by all middlewares and handlers throughout that request's lifecycle
- Automatically available — every request gets its own context

Every language/framework has its own implementation, but the concept is universal.

```
Request A → Context A { userId: 1, role: "admin", requestId: "abc-123" }  
Request B → Context B { userId: 7, role: "user", requestId: "def-456" }
```

These are isolated — middleware in Request A cannot read Context B.

Why it Exists

Without request context, passing data between middlewares would require directly threading values through function arguments, tightly coupling components. Context provides a **decoupled shared state**.

Common Uses

1. Auth data propagation

Authentication middleware verifies the token → extracts `userId`, `role`, `permissions` → stores in context. Downstream handlers read from context instead of trusting client-provided values.

```
Why not take userId from the request body?  
→ A malicious client could send userId of another user.  
→ Always use the userId from verified auth context.
```

2. Request ID (Distributed Tracing)

An early middleware generates a `UUID` and stores it as `requestId` in context. All subsequent logs, downstream service calls, and external API calls include this ID (e.g., in `X-Request-ID` header). Enables tracing a single request across multiple services in a microservice architecture.

3. Cancellation / Timeout Signals

Context can carry deadlines and abort signals. If the client disconnects or a timeout is exceeded, the signal propagates through context to downstream services, preventing

them from doing unnecessary work.

Context in the Auth → Handler Flow

Auth Middleware:

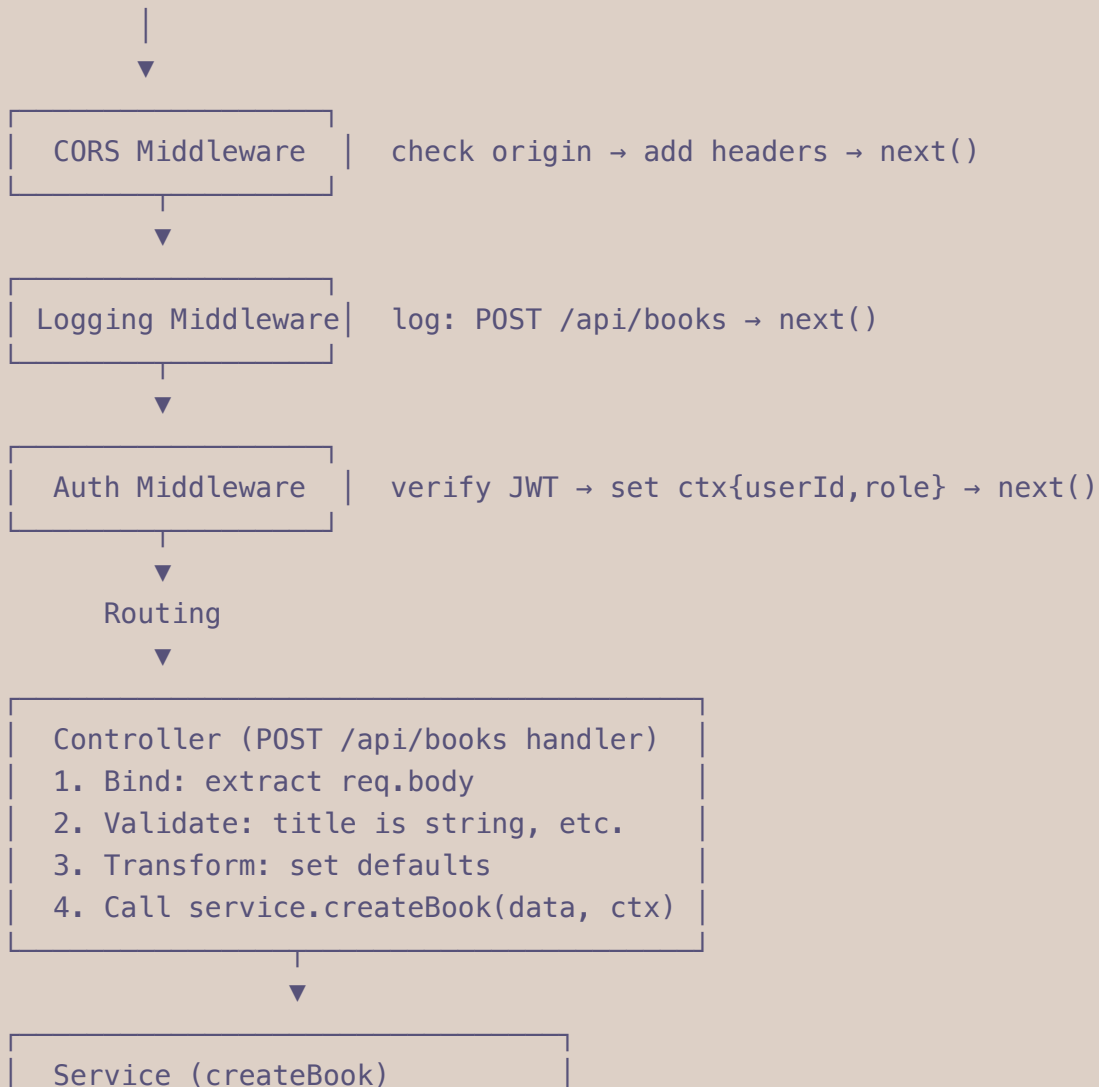
```
verifyToken(req.headers.authorization)
→ userId = 42, role = "admin"
→ ctx.set("userId", 42)
→ ctx.set("role", "admin")
→ next()
```

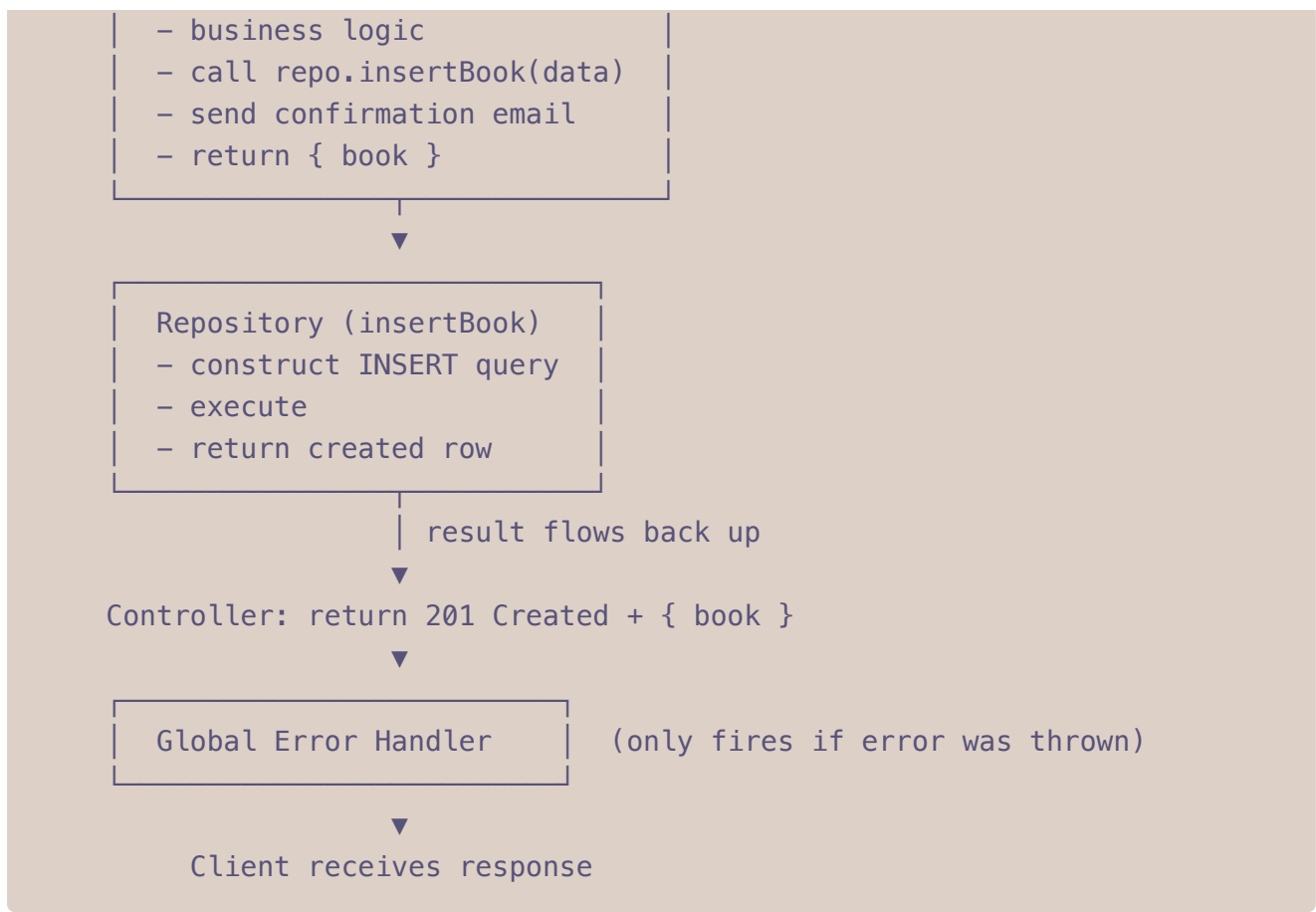
Handler (POST /books):

```
userId = ctx.get("userId") // 42 – trusted, from auth
role   = ctx.get("role")   // "admin"
// insert book with userId = 42
// never use req.body.userId – could be spoofed
```

5. Full Request Lifecycle — Annotated

Client: POST /api/books { title: "SICP", authorId: 3 }





Quick Revision Checklist

- ☐ Controller responsibilities: bind request data → validate/transform → call service → send response
- ☐ Controller is the only layer that deals with HTTP (status codes, req/res objects)
- ☐ Service layer: business logic, orchestration, external calls — no HTTP concepts
- ☐ Repository layer: one method = one DB operation; no business logic
- ☐ Middleware signature: `(request, response, next)` — `next()` passes to the next context
- ☐ Middleware short-circuits by sending a response and NOT calling `next()`
- ☐ Common middlewares: CORS, security headers, authentication, rate limiting, logging, global error handler
- ☐ Global error handler goes **last** — ordering of middlewares is critical
- ☐ Request context: per-request key-value store, accessible to all middlewares and handlers
- ☐ Always read `userId` from auth context, never from the request body — prevents spoofing
- ☐ Context use cases: auth data, request ID for distributed tracing, cancellation/timeout signals